

ЛЕКЦИЯ 8

Распределённые конечные автоматы в управлении робототехническими системами

От формальной модели FSM к сетям Петри, statecharts, поведенческим деревьям и верификации

две части по 45

«...ение распределёнными робототехническими системами»

План лекции

- 1 Дискретно-событийный взгляд на поведение робота
- 2 Конечный автомат: определение, DFA и NFA
- 3 Автоматы с выходом: Мили и Мур
- 4 Композиция автоматов и взаимодействующие автоматы (CFSM)
- 5 Распределённые автоматы: локальное и глобальное состояние

- 6 Сети Петри: параллелизм, конфликт, синхронизация, инварианты
- 7 Иерархические автоматы и statecharts Харела
- 8 Поведенческие деревья: отраслевой стандарт Nav2
- 9 Автомат складского робота и цикл исполнения
- 10 Верификация: model checking и логика LTL

Синие — первая часть · янтарные — вторая часть

ЧАСТЬ 1

Часть 1. Теория: автоматы, композиция, сети Петри

Формальный аппарат: чем описывают дискретное поведение робота и как складывают модели агентов в модель коллектива.

“

Робот — это не только траектории. Это конечный набор режимов и события, которые их переключают.

Дискретно-событийный взгляд на систему управления

Зачем нужен дискретно-событийный уровень

- Непрерывная динамика описывает как двигаться, автомат — что делать сейчас
- События: сообщение, срабатывание датчика, тайм-аут, завершение операции
- Внутри состояния работают регуляторы и планировщики — гибридная архитектура
- Логика режимов должна быть обозримой и проверяемой человеком
- Ошибки координации живут именно здесь, а не в ПИД-контурах

Факт

Большинство аварий подвижных роботов на складах — не ошибки регулятора, а ошибки логики режимов: робот поехал, когда не должен был, или не заметил, что задача уже отменена.

Формальное определение конечного автомата

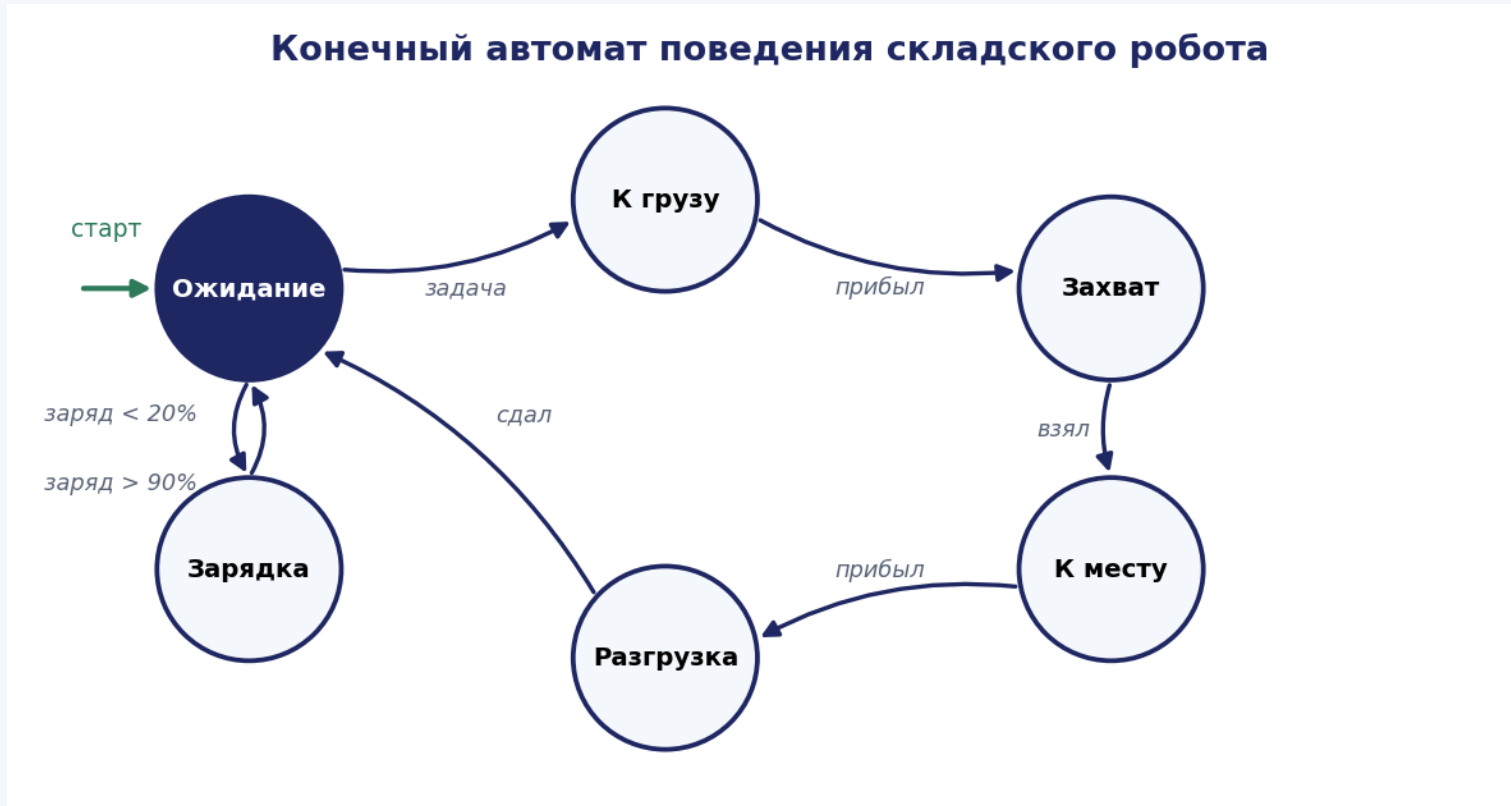
$$M = \langle Q, \Sigma, \delta, q_0, F \rangle, \quad \delta : Q \times \Sigma \rightarrow Q$$

- Q — конечное множество состояний: ожидание, движение, захват, зарядка
- Σ — входной алфавит событий: команда диспетчера, сигнал датчика, тайм-аут
- δ — функция переходов; q_0 — начальное состояние; F — заключительные состояния
- Теорема Клини: языки конечных автоматов = регулярные языки

Расширенная функция: $\delta(q, \epsilon) = q$, $\delta(q, wa) = \delta(\delta^*(q, w), a)$

1

Автомат поведения складского робота



Состояния: Ожидание (начальное), К грузу, Захват, К месту, Разгрузка, Зарядка. Переходы помечены событиями: «задача», «прибыл», «взял», «сдал», «заряд < 20%», «заряд > 90%». Вся логика поведения робота умещается в один читаемый граф.

- DFA: пара (состояние, символ) однозначно задаёт следующее состояние
- NFA: $\delta: Q \times \Sigma \rightarrow 2^Q$, допускаются ϵ -переходы; удобен для спецификации
- Рабин–Скотт: NFA и DFA эквивалентны, детерминизация даёт до 2^n состояний
- Глобальная модель PPTC недетерминирована даже при детерминированных агентах
- Причины: порядок доставки сообщений, скорости агентов, моменты событий

Мур: $\lambda: Q \rightarrow \Lambda$

- Выход зависит только от состояния
- Держится, пока автомат в состоянии
- Реакция с задержкой на такт
- Режимные выходы: «двигатели включены»
- Устойчив к дребезгу входа

Мили: $\lambda: Q \times \Sigma \rightarrow \Lambda$

- Выход рождается в момент перехода
- Немедленная реакция в том же такте
- Как правило, меньше состояний
- Событийные выходы: «отправить пакет»
- Ложное событие — ложный выход



Как думаете?

Дребезг контакта даёт серию ложных событий на входе. Какая модель автомата пострадает сильнее?

А

Мур — выход привязан к состоянию

В

Мили — выход рождается на переходе

С

Обе одинаково: они эквивалентны

Д

Ни одна: дребезг фильтруется автоматом сам

? Как думаете?

Дребезг контакта даёт серию ложных событий на входе. Какая модель автомата пострадает сильнее?

- A** Мур — выход привязан к состоянию
- B** Мили — выход рождается на переходе
- C** Обе одинаково: они эквивалентны
- D** Ни одна: дребезг фильтруется автоматом сам

ОТВЕТ

Мили. Каждое ложное событие порождает выходное воздействие. У Мура выход держится состоянием, и лишний импульс не даёт лишней команды.

$$Q = Q_1 \times Q_2 \times \dots \times Q_N, \quad q = (q^1, \dots, q^N)$$

- Синхронное произведение: все компоненты шагают одновременно, общий такт
- Асинхронное (interleaving): в каждый момент шагает ровно один компонент
- Параллельная композиция $M_1 \parallel M_2$: общие события — только одновременно (рандеву)
- Композиция $\parallel$ ассоциативна и коммутативна — модель из любого числа агентов

$$10^{10}$$

глобальных состояний у 10 роботов по 10 состояний

$$2^n$$

рост при детерминизации NFA в худшем случае

$$n \cdot k$$

дуг сквозных событий в плоском автомате

- Размер композиции растёт экспоненциально по числу агентов
- Борьба со взрывом: иерархия при проектировании, символьные методы при анализе

- Автоматы связаны однонаправленными FIFO-каналами (Бранд, Зафиропуло)
- Переход $!m$ — отправка в канал; переход $?m$ — приём из головы очереди
- Глобальное состояние = вектор локальных состояний + содержимое каналов
- Неограниченные очереди $\Rightarrow$ мощность машины Тьюринга, анализ неразрешим
- Практика: модели с ограниченными очередями и обработкой потерь

Пример

Протокол stop-and-wait: передатчик Idle $\rightarrow$ Sending по $!data$, Sending $\rightarrow$ Idle по $?ack$, Sending $\rightarrow$ Sending по тайм-ауту. Проверяемое свойство: каждый кадр доставляется ровно один раз даже при потерях.

- Декомпозиция: глобальную спецификацию проецируют на агентов (projection)
- Композиция: локальные автоматы проектируют по ролям — так делают чаще
- Глобальное состояние не известно достоверно никому — только оценки
- Синхронизация: heartbeat, событийная рассылка, консенсус, рандеву
- «Безопасно при устаревании»: рассогласование ведёт к остановке, не к аварии

Ключевая идея

Для мягкой координации хватает периодической рассылки состояний; жёсткая (совместный подъём груза двумя манипуляторами) требует рандеву или двухфазного подтверждения.

$$N = \langle P, T, F, W, M_0 \rangle, \quad M : P \rightarrow \mathbb{N}_0$$

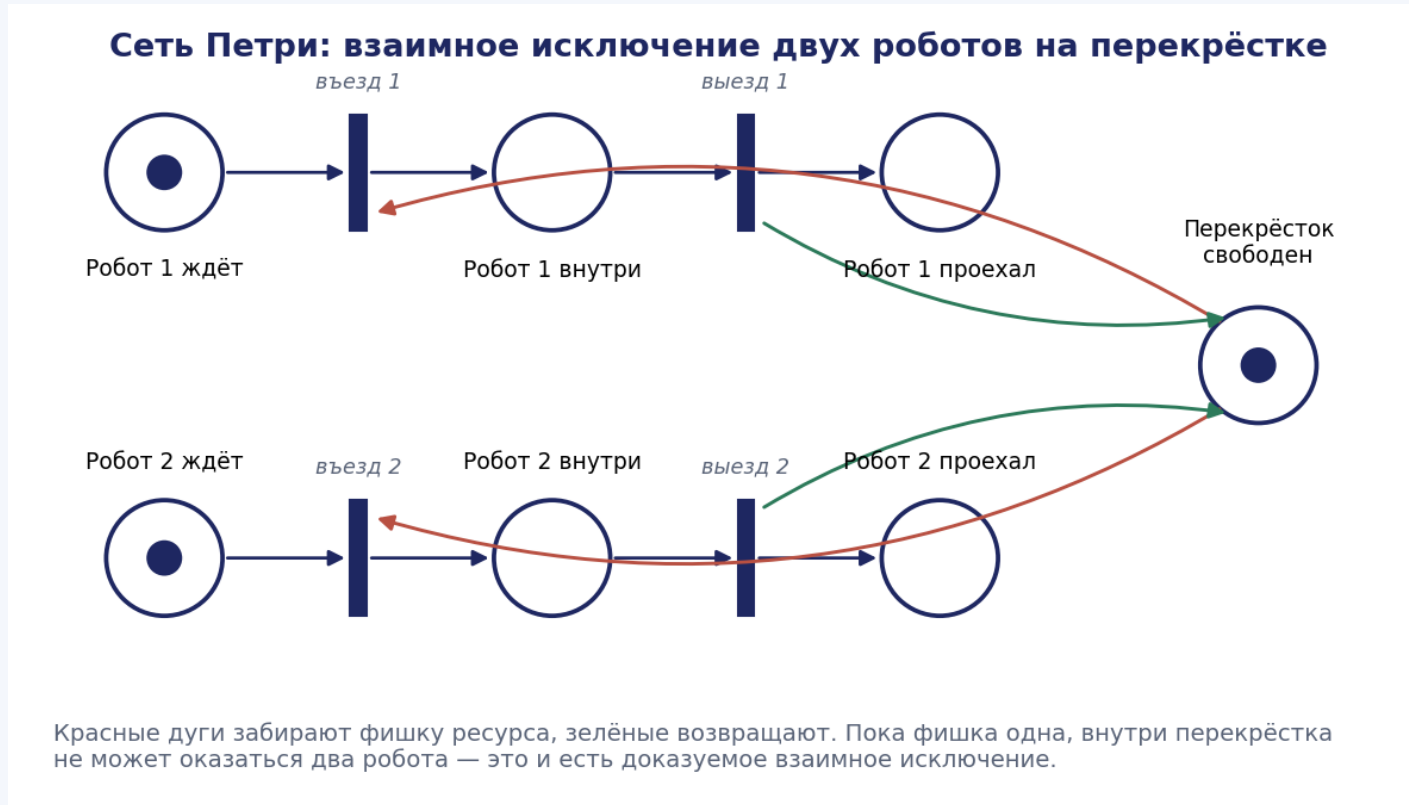
- P — позиции (условия, ресурсы), T — переходы (события), $P \cap T = \emptyset$
- F — дуги «позиция $\rightarrow$ переход» и «переход $\rightarrow$ позиция», W — их кратности
- Маркировка M — распределение фишек; переход возбуждён при достатке фишек
- Состояние распределено: фишки движутся одновременно и независимо

3 Правило срабатывания перехода



Три базовых паттерна: параллелизм, конфликт, синхронизация (рандеву)

3 Сеть Петри: взаимное исключение на перекрёстке



У каждого робота цепочка позиций «ждёт → внутри → проехал». Общая ресурсная позиция «Перекрёсток свободен» содержит ровно одну фишку. Красные дуги забирают фишку при въезде, зелёные возвращают её при выезде.

$$y^T \cdot C = 0 \Rightarrow y^T \cdot M = y^T \cdot M_0$$

- C — матрица инцидентности сети, y — вектор-инвариант позиций
- Для перекрёстка: $M(p_2) + M(p_5) + M(p_7) = 1$ при любой достижимой маркировке
- Отсюда $M(p_2)$ и $M(p_5)$ не могут одновременно равняться единице
- Взаимное исключение доказано алгеброй, а не обходом графа достижимости

- Достижимость: попадёт ли сеть в маркировку M ? Разрешима (Майр), но крайне сложна
- Ограниченность: $\leq k$ фишек в позиции; 1-ограниченная сеть называется безопасной
- Ограниченность = нет переполнения буферов; дерево покрытия Карпа–Миллера
- Живость: любой переход возбудим из любой достижимой маркировки — нет тупиков
- Обратимость — возврат в M_0 ; консервативность — сохранение ресурсов

3

Расширения сетей Петри: что за что платит

Расширение	Что добавляет	Цена
Временные (Мерлин)	интервалы задержек переходов	сложнее анализ, но разрешим
Раскрашенные (Йенсен)	типизированные фишки «робот 1/2»	компактность против наглядности
Ингибиторные дуги	запрет при наличии фишки	мощность МТ, анализ неразрешим
Иерархические	вложенные подсети	нужна дисциплина интерфейсов

Выбор расширения — компромисс между адекватностью модели и возможностью её проанализировать.



Проверим понимание

В сети перекрёстка ресурсную позицию убрали, оставив цепочки движения роботов. Что произойдёт?

А

Сеть станет неограниченной

В

Оба робота смогут оказаться в зоне одновременно

С

Сеть потеряет живость и встанет в тупик

Д

Ничего не изменится: цепочки независимы



Проверим понимание

В сети перекрёстка ресурсную позицию убрали, оставив цепочки движения роботов. Что произойдёт?

A

Сеть станет неограниченной

B

Оба робота смогут оказаться в зоне одновременно

C

Сеть потеряет живость и встанет в тупик

D

Ничего не изменится: цепочки независимы

ОТВЕТ

Оба окажутся в зоне: без общей входной позиции переходы въезда перестают конфликтовать, и Р-инвариант взаимного исключения исчезает.

П Е Р Е Р Ы В

Конец первой части

Разобрали формальный аппарат: автомат, композиция, CFSM, сети Петри и инварианты. После перерыва — инженерия: иерархия, поведенческие деревья, реальный автомат складского робота и верификация.

ЧАСТЬ 2

Часть 2. Инженерия: иерархия, деревья, верификация

Как масштабировать описание поведения, чем его заменяет индустрия и как доказать, что координация корректна.

- Сквозное событие «критический разряд» нужно провести из каждого состояния
- n состояний $\times k$ сквозных событий = $n \cdot k$ однотипных дуг
- Добавили состояние — дописали k дуг; забыли одну — получили дыру в логике
- Диаграмма перестаёт читаться раньше, чем перестаёт работать
- Параллельные аспекты дают декартово произведение состояний

Кейс

Робот с 12 рабочими состояниями и 4 сквозными событиями (разряд, отказ, стоп-кнопка, отмена задачи) требует 48 дополнительных дуг. Ревью такой диаграммы человеком фактически невозможно.

- Формула: конечные автоматы + глубина + ортогональность + широковещение
- Глубина: переход из суперсостояния наследуется подсостояниями — 1 дуга вместо n
- История H: возврат к прерванной задаче после подзарядки
- Ортогональность: регионы активны одновременно — 6 состояний вместо $2 \cdot 2 \cdot 2$
- Run-to-completion, guards, entry/exit/do — семантика Мили плюс Мур

- Верхние уровни — фазы миссии, синхронизируются редкими сообщениями
- Нижние уровни — моторика и локальные реакции, работают автономно
- Сетевой обмен пропорционален частоте переключений верхнего уровня
- Отказ связи не парализует нижние уровни — плавная деградация
- Реализации: SMACH, пакеты ROS 2, Stateflow, инструменты UML

Ключевая идея

Иерархия автомата задаёт не только структуру кода, но и структуру трафика: чем ниже уровень, тем меньше он нуждается в сети. Это прямой рецепт устойчивости к потере связи.

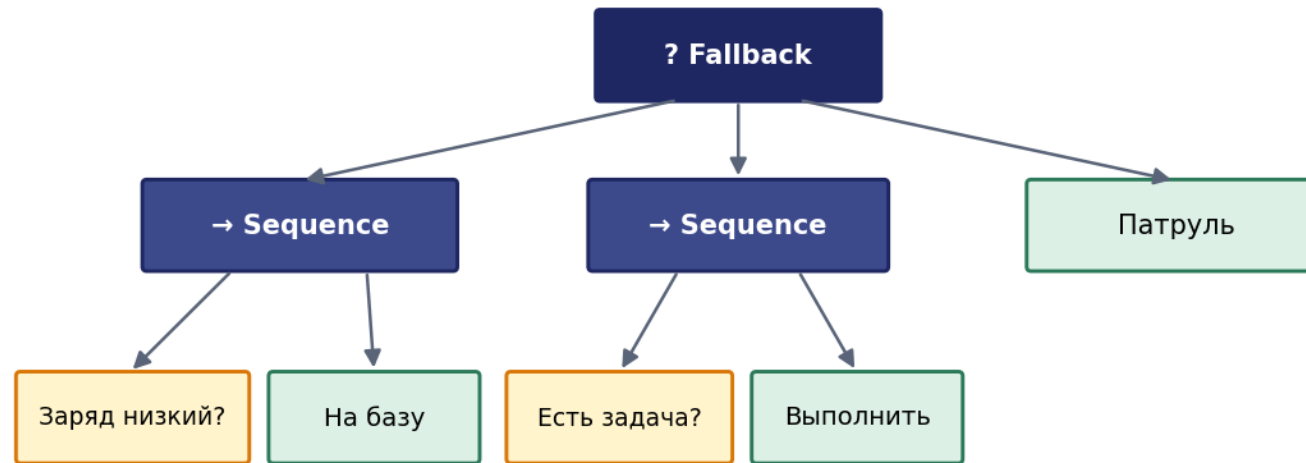
“

Индустрия ушла от рукописных автоматов к поведенческим деревьям — не потому, что автоматы неверны, а потому что они не складываются.

Nav2, ArduPilot, современный геймдев

- Дерево обходится **тиками** от корня с частотой 10–100 Гц
- Каждый узел возвращает статус: **Success**, **Failure** или **Running**
- Sequence — И: идёт по детям, пока Success; первый Failure прерывает
- Fallback (Selector) — ИЛИ: пробует детей, пока не получит Success
- Листья: Condition (проверка) и Action (действие, может быть Running)

Поведенческое дерево: та же логика, но модульно и переиспользуемо



Fallback пробует ветви слева направо до первого успеха; Sequence требует успеха всех потомков. Жёлтые — условия, зелёные — действия. Добавить поведение = вставить поддерево, не трогая остальное.

Корень Fallback перебирает ветви слева направо. Первая ветвь Sequence: условие «Заряд низкий?» → действие «На базу». Вторая: условие «Есть задача?» → действие «Выполнить». Если обе ветви дали Failure, срабатывает запасное действие «Патруль».

5

Один тик поведенческого дерева



Реактивность бесплатна: приоритеты перепроверяются на каждом тике

- Decorator — узел с одним ребёнком, меняющий его поведение
- Inverter, Retry(n), Timeout(t), RateController, ForceSuccess
- Blackboard — общая память дерева: цель, поза, уровень заряда
- Условия читают blackboard, действия его пишут — связь без лишних дуг
- Поддерево — единица переиспользования: «доехать до точки» один раз и везде

Пример

В Nav2 стандартное дерево навигации оборачивает вычисление пути в RecoveryNode: при неудаче запускается ветвь восстановления — очистка карты стоимости, разворот на месте, отход назад, затем повторная попытка.

Конечный автомат

- Число переходов растёт $\sim$ квадратично
- Состояние явное — легко логировать
- Есть строгая формальная семантика
- Верифицируется model checking
- Правка поведения трогает много дуг

Поведенческое дерево

- Расширение = вставка поддеревя
- Приоритет виден как порядок детей
- Реактивность из коробки, через тики
- Поддеревья переиспользуемы
- Верификация сложнее и менее зрелая

Задача	Формализм	Почему
Логика задач робота, Nav2	поведенческое дерево	реактивность, вставка поддеревьев
Сложное поведение одного агента	statechart	иерархия и ортогональные регионы
Протокол обмена сообщениями	CFSM	явные очереди, верифицируется в SPIN
Конкуренция за ресурсы	сеть Петри	инварианты, живость, безопасность
Сертифицируемый контур безопасности	плоский FSM	малый размер, полная верификация

Отладка BT: Groot2 показывает дерево вживую, подсвечивая тикаемые узлы и их статусы, и умеет проигрывать записанный лог.



Вопрос на понимание

В поведение робота нужно добавить аварийную остановку по стоп-кнопке с наивысшим приоритетом. Что дешевле?

A

FSM: добавить одно состояние Stop

B

FSM: провести дуги из всех состояний

C

BT: вставить ветвь первой слева под корневым Fallback

D

Одинаково по трудоёмкости



Вопрос на понимание

В поведение робота нужно добавить аварийную остановку по стоп-кнопке с наивысшим приоритетом. Что дешевле?

A

FSM: добавить одно состояние Stop

B

FSM: провести дуги из всех состояний

C

BT: вставить ветвь первой слева под корневым Fallback

D

Одинаково по трудоёмкости

ОТВЕТ

BT: одна вставка поддерева, приоритет задаётся позицией. В плоском FSM нужны дуги из каждого состояния — n правок вместо одной.

6 Автомат складского робота: функция переходов

Состояние	task	arrived	low_batt	иное
Idle	GoToPick	—	GoToCharge	—
GoToPick	—	Pick	GoToCharge	—
Pick	—	—	— (игнор)	loaded → GoToDrop
GoToDrop	—	Drop	— (игнор)	—
Drop	—	—	—	unloaded → Idle
Charge	—	—	—	charged → Idle
любое	—	—	—	error → Fault

$Q = \{Idle, GoToPick, Pick, GoToDrop, Drop, GoToCharge, Charge, Fault\}$; low_batt игнорируется с грузом на борту — это политика безопасности склада.

6

Цикл исполнения автомата в контроллере



Run-to-completion: событие обрабатывается целиком до извлечения следующего

- Спецификация протоколов: контрактная сеть — Объявление → Заявки → Присуждение
- Совместные действия: randevу «оба готовы к подъёму» как общее событие ||
- Разделяемые ресурсы: арбитр Free/Busy(i), схема Request → Grant → Release
- Реализация семафора: координатор зоны либо токен (Рикарт–Агравала)
- При отказе связи автоматы работают локально — постепенная деградация

- Кларк, Эмерсон, Сифакис, премия Тьюринга 2007:
алгоритмическая проверка $M \models \phi$
- Тестирование распределённой системы принципиально неполно
- Нарушение свойства даёт контрпример — конкретную трассу к тупику
- Инструменты: SPIN/Promela, NuSMV, UPPAAL для временных автоматов
- Против взрыва состояний: BDD, редукция частичных порядков, абстракция

Ключевая идея

Тест показывает наличие ошибки в одном исполнении. Верификация доказывает её отсутствие во всех исполнениях модели. Для редких чередований событий это принципиально разные гарантии.

$$G\neg(\text{зона}_1 \wedge \text{зона}_2), \quad G(\text{запрос}_1 \rightarrow F \text{ доступ}_1)$$

- $X \phi$ — в следующем состоянии, $F \phi$ — когда-нибудь, $G \phi$ — всегда, $\phi U \psi$ — до тех пор
- Безопасность: никогда не случится плохого — оба робота в зоне одновременно
- Живучесть: хорошее обязательно случится — запрос будет удовлетворён, нет голодания
- Справедливость: $G F$ выполняется₁ — робот бесконечно часто получает возможность

2–3

агента в практически
верифицируемой абстракции

линейно

по размеру модели при
проверке LTL

ЭКСПОН.

по длине темпоральной
формулы

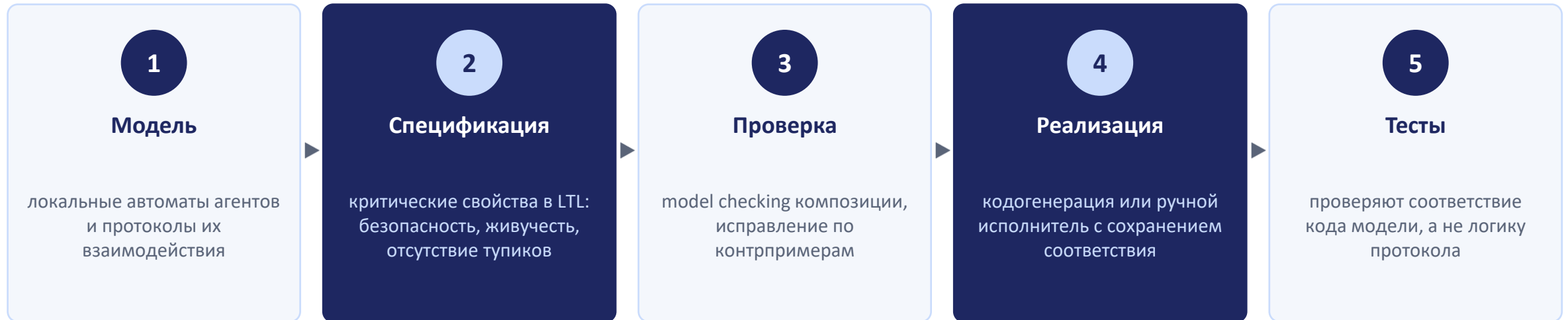
N

параметризованная
верификация неразрешима в
общем

- Верифицируют не систему, а её дискретную абстракцию — протокол координации
- Масштабирование на любое N доказывают отдельно либо принимают инженерно

7

Место верификации в разработке



Логические ошибки координации исключены до первого выезда робота

Выводы

- 1 КА $M = \langle Q, \Sigma, \delta, q_0, F \rangle$ — базовая модель дискретно-событийного поведения робота
- 2 Мили и Мур эквивалентны по силе, но различаются реакцией и устойчивостью к шуму
- 3 Композиция локальных автоматов растёт экспоненциально по числу агентов
- 4 Сети Петри выражают параллелизм и конфликт; Р-инварианты доказывают исключение
- 5 Statecharts дают иерархию, ортогональность и уровни координации по трафику
- 6 Поведенческие деревья масштабируются вставкой поддеревя — стандарт Nav2
- 7 FSM остаётся базой верификации: безопасность и живучесть проверяются в LTL

Вопросы для самопроверки

1. Определите DFA $M = \langle Q, \Sigma, \delta, q_0, F \rangle$ и поясните компоненты для системы управления роботом
2. Чем автомат Мили отличается от автомата Мура? Где предпочтительна каждая модель?
3. В чём различие синхронного и асинхронного производства автоматов?
4. Как через композицию с общими событиями реализуется randevu двух роботов?
5. Почему анализ CFSM с неограниченными очередями неразрешим? Практический вывод?
6. Как в сети Петри выражаются параллелизм, конфликт и синхронизация?
7. Постройте Р-инвариант сети перекрёстка и выведите из него взаимное исключение
8. Какие три механизма statecharts устраняют комбинаторный рост плоских автоматов?
9. Назовите узлы поведенческого дерева и поясните роль статуса Running
10. Сравните FSM и BT по масштабируемости, реактивности и верифицируемости
11. Запишите в LTL взаимное исключение и отсутствие голодания; их классы?